



Lập trình hướng đối tượng

Giới thiệu về Lập trình hướng đối tượng

Nội dung



- Lịch sử phát triển của kỹ thuật lập trình
- Hạn chế của kỹ thuật lập trình truyền thống
- Khái niệm lập trình hướng đối tượng
 - Đóng gói/Che giấu thông tin

Tài liệu tham khảo



- *Giáo trình Lập trình HĐT, chương 3*

Tổng quan về NNLT



- <https://www.youtube.com/watch?v=Og847HVwRSI>

Ngôn ngữ lập trình tốt?

- Khả năng thể hiện (Expressive power): Phần lớn các ngôn ngữ là Turing-complete. Nhưng ngôn ngữ tốt sẽ giúp cho lập trình viên viết các đoạn mã rõ ràng, chính xác, dễ bảo trì (đặc biệt cho các hệ thống lớn)
- Dễ cho người mới học
- Dễ cài đặt: chạy trên máy nhỏ, cài đặt nền tảng dễ dàng, miễn phí
- Chuẩn hóa
- **Mã nguồn mở (trình biên dịch/thông dịch)**
- **Có hệ sinh thái tốt => Java**

Phân loại NNLT

- Declarative (khai báo)
 - Functional: Lisp/Scheme, ML, Haskell, Erlang
 - Logic, constraint-based: Prolog, SQL
- Imperative (mệnh lệnh)
 - Procedural: C, Ada, Fortran, . . .
 - Object-oriented: Smalltalk, Eiffel, Java, . . .

Lập trình thủ tục



```
int gcd(int a, int b){  
    while(b > 0){  
        int c = a % b;  
        a = b;  
        b = c;  
    }  
    return a;  
}
```

Lập trình hàm (lập trình chức năng)



```
1  -module(helloworld).
2  -export([gcd/2,start/0]).
3
4  gcd(A,B) when A == 0 ->B, io:fwrite("~w~n",[B]);
5  gcd(A,B) when B == 0 ->A, io:fwrite("~w~n",[A]);
6  gcd(A,B) when A == B ->A, io:fwrite("~w~n",[A]);
7  gcd(A,B) when A > B -> gcd(A-B,B);
8  gcd(A,B) when B > A -> gcd(A,B-A).
9
10
11  start() ->gcd(18,63).|
```


Lập trình logic

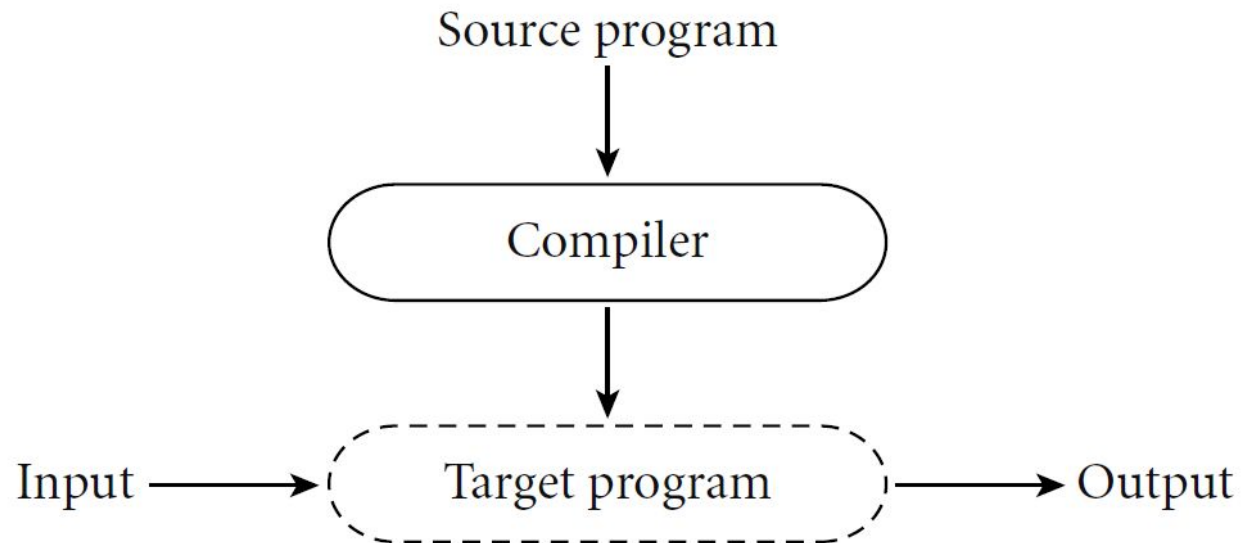


```
gcd(X,Y,Z) :- X=0, Z=Y.  
gcd(X,Y,Z) :- Y=0, Z=X.  
gcd(X,Y,Z) :- X=Y, Z=X.  
gcd(X,Y,Z) :- X>Y, X1 is X-Y, gcd(X1,Y,Z).  
gcd(X,Y,Z) :- Y>X, Y1 is Y-X, gcd(X,Y1,Z).|
```

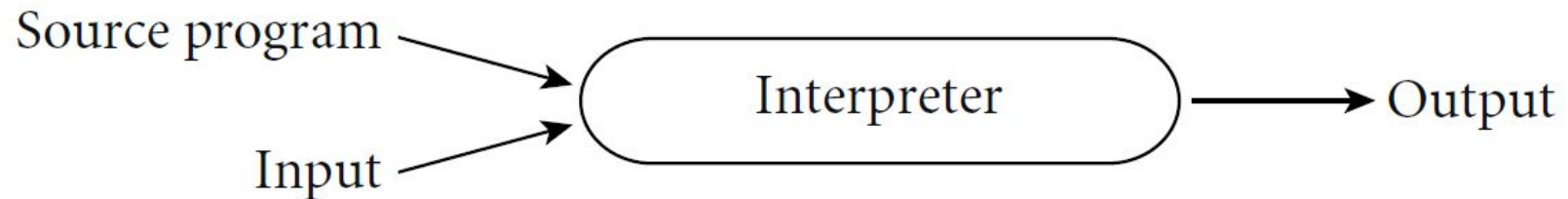
```
?-gcd(18,63,9)
```

```
?-gcd(15,25,X)
```

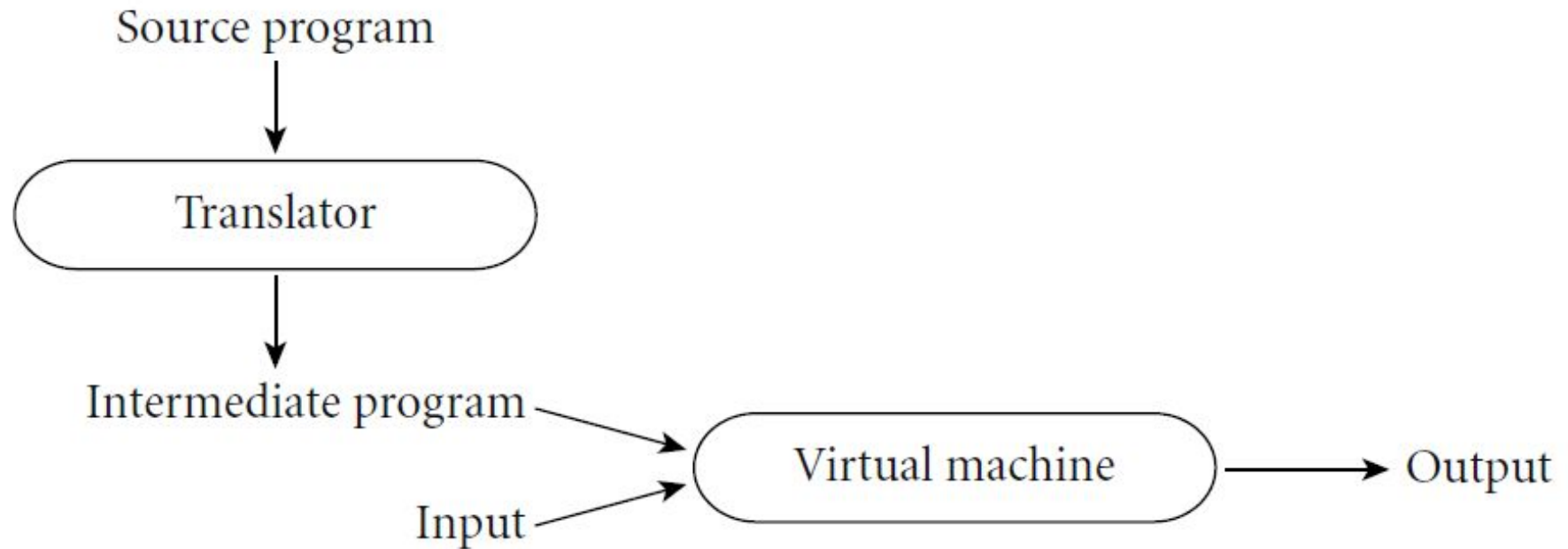
Biên dịch và thông dịch



Biên dịch và thông dịch



Biên dịch và thông dịch



Phần mềm ngày càng phức tạp



- Kích thước ngày càng lớn*
 - LibreOffice: 9.5M dòng lệnh
 - Chromium: 25.6M dòng lệnh
 - NetBeans IDE: 95.4M dòng lệnh
- Số lượng người tham gia phát triển lớn
- Người dùng ngày càng đòi hỏi nhiều chức năng => phần mềm luôn cần được sửa đổi

Nguồn: <https://www.openhub.net>, 8/2020

Dữ liệu trong lập trình thủ tục



```
struct MyDate {  
    int year, month, day;  
};  
...  
print_date(MyDate d) {  
    printf("%d / %d / %d\n", d.day,  
        d.month, d.year);  
}
```

Dữ liệu trong lập trình thủ tục



```
struct MyDate {  
    int year; int month; int day;  
}
```

```
MyDate d;  
d.day = 32; // invalid day  
  
d.day = 31; d.month = 2; // how to check?  
d.day = d.day + 1; //
```

Dữ liệu trong lập trình thủ tục



Thay đổi cấu trúc dữ liệu:

```
struct MyDate {  
    int year, month, day;  
}
```



```
struct MyDate {  
    public short year;  
    public short mon_n_day;  
}
```


Giải pháp



- Che giấu dữ liệu (che giấu cấu trúc)
- Truy cập dữ liệu thông qua giao diện xác định

```
class MyDate {  
    private int year, mon, day;  
    public int getDay() {...}  
    public boolean setDay(int) {...}  
    ...  
}
```

Sử dụng giao diện



MyCalendar.java:

```
MyDate d = new MyDate();
```

```
...
```

```
d.day = 32; // error
```

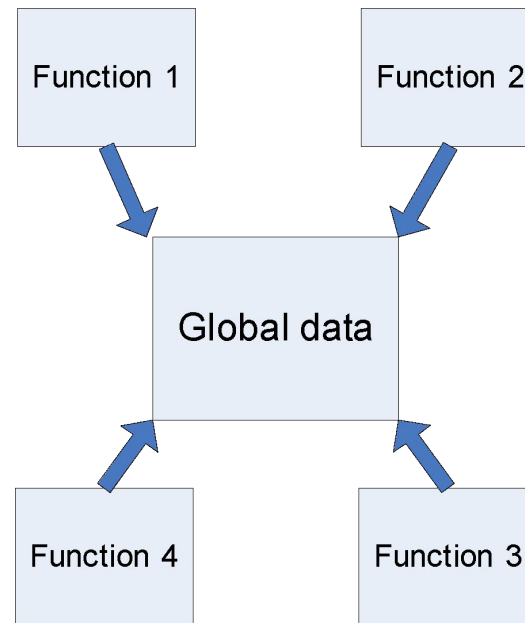
```
d.setDay(31);
```

```
d.setMonth(2); // should return False
```

What is method?



```
d.setDay(31);
```



Đóng gói và che giấu thông tin



- Đóng gói: Đóng gói dữ liệu và các thao tác tác động lên dữ liệu thành một thể thống nhất (lớp đối tượng) thuận tiện cho sử dụng lại
- Che giấu thông tin
 - thao tác với dữ liệu thông qua các giao diện xác định
 - che giấu cấu trúc dữ liệu khỏi đoạn mã sử dụng

Lớp và đối tượng

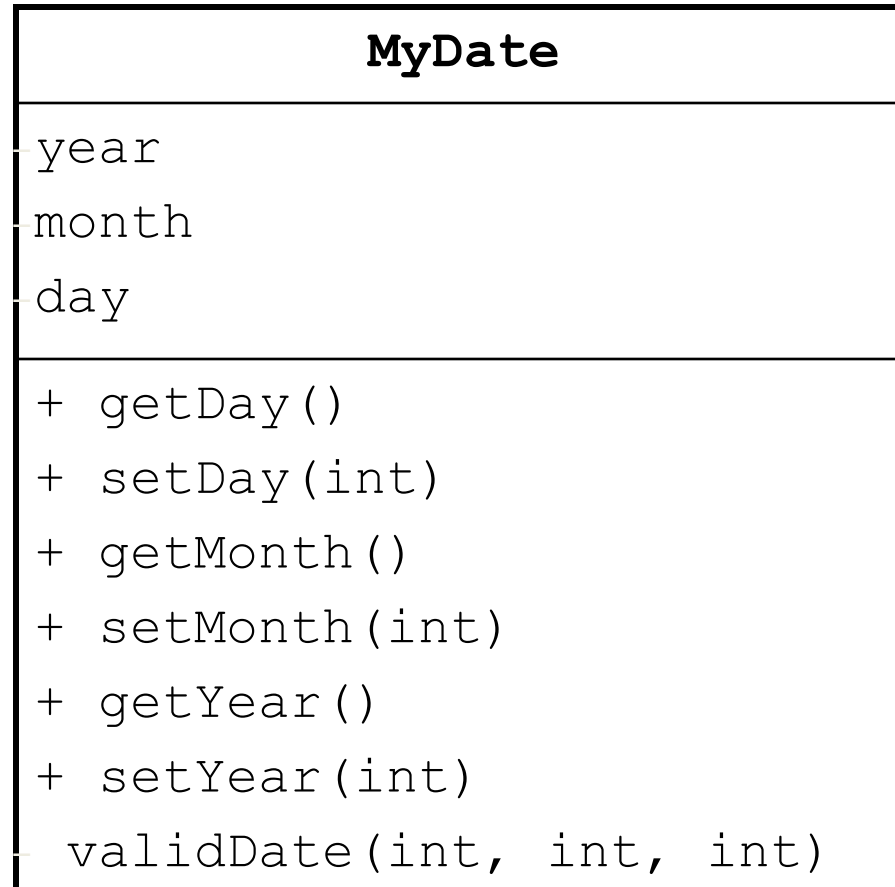
- Lớp đối tượng (class) là khuôn mẫu để sinh ra đối tượng
- Đối tượng là thể hiện (instance) của một lớp. Đối tượng có
 - thuộc tính (dữ liệu)
 - hành vi (phương thức)

Hệ thống hướng đối tượng



- Bao gồm một tập các đối tượng
 - mỗi đối tượng chịu trách nhiệm một công việc
- Các đối tượng tương tác thông qua trao đổi thông điệp (*message passing*)
- Các đối tượng có thể tồn tại phân tán/có thể hoạt động song song

Mô hình hóa đối tượng



Field

Method

Lợi ích của lập trình hướng đối tượng



- năng suất lập trình (năng suất phát triển)
- chất lượng phần mềm
- tính hiệu được của phần mềm
- vòng đời của phần mềm

OOP và OOL (Object Oriented Language)



- Có thể thể hiện phần nào tư tưởng đóng gói/che giấu thông tin trên ngôn ngữ thủ tục
 - không triệt để, khó kiểm soát
- Ngôn ngữ hướng đối tượng cung cấp khả năng kiểm soát truy cập; ngoài ra
 - kế thừa
 - đa hình

Lập trình hướng đối tượng là tốt nhất?

